Introduction à la Programmation Orientée Objet en Python

Rédigé par : Fabien Poirier
Supervisé par : Geoffrey Groff

February 6, 2025

1. Les méthodes et la documentation
2. Les importations
3. L'avant POO
4. La programmation orientée objet

Plan du Chapitre

1. Les méthodes et la documentation
   - Les méthodes : procédure / fonction
   - Les passages de paramètres
   - Les différents types de retour
   - La documentation
2. Les importations
3. L'avant POO
4. La programmation orientée objet

Les méthodes

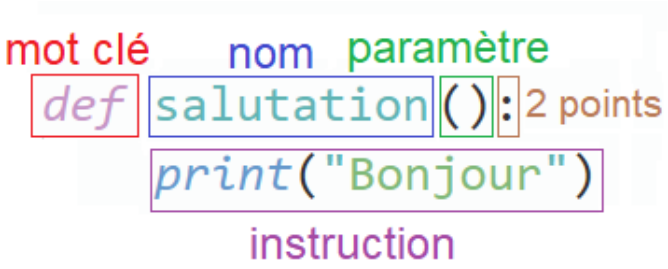

Figure: Procédure

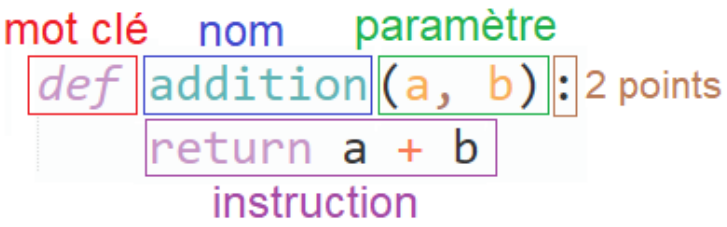

Figure: Fonction

- Lorsque vous utilisez le mot-clé **def**, suivi d'un nom et d'un ensemble de paramètres entre parenthèses, vous déclarez une méthode.
- Pour rappel, la différence entre une fonction et une procédure se situe au niveau du retour de valeur : les procédures n'en ont pas, contrairement aux fonctions.

Les paramètres optionnels

paramètre optionel

```
def saluer(nom="tout le monde"):
    print("Bonjour,", nom)

# Appel de la fonction sans spécifier de paramètre
saluer()  # Output: Bonjour, tout le monde

# Appel de la fonction en spécifiant un paramètre
saluer("Vincent Matias") # Output : Bonjour, Vincent Matias
```

prototype

- Pour définir un paramètre optionnel, il suffit de lui attribuer une valeur par défaut lors de la déclaration du prototype.

✓ Les paramètres optionnels doivent toujours être placés après les autres paramètres.

Les paramètres optionnels (2)

- Définition de la méthode

```
def saluer(nom, salutation="Bonjour", ponctuation="!"):
    print(salutation + ", " + nom + ponctuation)
```

- Appel en utilisant les valeurs par défaut

```
saluer("Armand") # Output: Bonjour, Armand!
```

- Appel en précisant une valeur pour ponctuation

```
saluer("Émile", ponctuation=".") # Output: Bonjour, Émile.
```

- Appel en précisant des valeurs pour chaque paramètres

```
saluer("Mariam", "Coucou", ".") # Output: Coucou, Mariam.
```

- Si vous avez plusieurs paramètres optionnels mais que vous souhaitez uniquement changer la valeur de l'un d'entre eux, vous devrez le spécifier via la syntaxe **nom_param=valeur** lors de l'appel de la méthode.

Passage par copie vs passage par référence

Passage par copie ?

Passage par référence ?

Mais que veulent dire ces termes ?

Passage par copie / Passage par référence

Passage par copie [X]

Une copie de la valeur de l'objet est passée à la méthode.
Cela signifie que toute modification de l'objet à l'intérieur de la méthode n'affectera pas l'objet original en dehors de la méthode.
Types associés : **int**, **float**, **complexe**

Passage par référence [✓]

Une référence à l'objet est passée à la méthode.
Cela signifie que toute modification de l'objet à l'intérieur de la méthode affectera également l'objet original en dehors de la méthode.
Type associés : **Collection, Itérable, Objet, Autre type...**

Le multi return !

- Définition de la fonction

```
def division_et_reste(a, b):
    quotient = a // b
    reste = a % b
    return quotient, reste
```

- Appel de la fonction

variables retournées — appel de la fonction

```
q, r = division_et_reste(10, 3)
print("Quotient:", q) # Affiche: Quotient: 3
print("Reste:", r)    # Affiche: Reste: 1
```

- En Python, une fonction peut renvoyer plusieurs valeurs à la fois sous forme de tuple. Vous pouvez stocker ces valeurs dans des variables distinctes en les séparant par des virgules, ce qui est appelé "unpacking".

Le caractère underscore (_)

- Définition de la fonction

```
def diviser_et_rester(a, b):
    quotient = a // b
    reste = a % b
    return quotient, reste
```

- Appel de la fonction

valeur stockée — valeur ignorée — appel de la fonction

```
q, _ = division_et_reste(10, 3)
print("Quotient:", q) # Affiche: Quotient: 3
```

- Lors de l'affectation des valeurs retournées, vous pouvez également ignorer certaines valeurs en les remplaçant par un underscore (_), indiquant que vous n'avez pas l'intention d'utiliser ces valeurs.

unpacking de valeur

```
def division_et_reste(a, b):
    quotient, reste = a // b, a % b
    return quotient, reste
```

Attention

- En Python, comme dans d'autres langages, il est tout à fait possible d'effectuer ce genre d'instruction. Cependant, il est toujours préférable de privilégier la lisibilité du code !

La méthode Main

```
if __name__ == "__main__":
    print("Hello World !")
```

Pourquoi cette structure ?

L'utilisation de `if __name__ == '__main__'` permet de rendre un fichier Python à la fois exécutable en tant que programme principal et importable comme module, en séparant clairement le code destiné à chaque cas d'utilisation.

return / yield ?

Les générateurs

Un générateur est une fonction spéciale qui permet de produire une séquence de valeurs de manière paresseuse, c'est-à-dire qu'il génère les valeurs au fur et à mesure de leur demande plutôt que de les calculer et de les stocker en mémoire en une seule fois. Cela permet d'économiser de la mémoire, en particulier lorsque vous travaillez avec de grandes séquences de données.

```
def generateur(start=0, end=100, step=1):
    for x in range(start, end, step):
        yield x
```

```
gen = generateur()
for valeur in gen :
    print(valeur)
```

Avantages des générateurs ! (1/2)

- Syntaxe : ils sont définis comme des fonctions, mais utilisent l'instruction yield au lieu de return. L'utilisation de yield permet de suspendre temporairement l'exécution et de la reprendre là où elle s'était arrêtée lors du prochain appel.
- Paresseux : ils ne calculent et ne renvoient les valeurs qu'au moment de leur appel, ce qui évite de stocker toute la séquence en mémoire.
- Itérable : ce sont des objets itérables **une seule fois** de par leur nature.

Avantages des générateurs ! (2/2)

- Efficacité : ils sont plus adaptés que les listes ou les tuples lorsqu'il s'agit de manipuler de grandes séquences de données, car ils ne chargent en mémoire qu'une seule valeur à la fois. De plus, ils sont responsables de leur mnémonique en cas d'implémentation.
- Génération infinie : ils peuvent être utilisés pour générer des séquences infinies de valeurs.

Les méthodes propre aux générateurs : send / close

- **send(value)** est utilisée pour envoyer une valeur à l'intérieur du générateur et reprendre son exécution jusqu'au prochain **yield**. La valeur envoyée est récupérée par le **yield** et peut être assignée à une variable dans le générateur.
- **close()** est utilisée pour arrêter définitivement l'exécution du générateur. Une fois appelée, tous les appels suivants à **next()** ou **send()** lèveront une exception **StopIteration**.

```
def generate_numbers():
    while True:
        received = yield
        print("Received:", received)

gen = generate_numbers()
next(gen)   # Démarrage du générateur
gen.send(1) # Envoie de la valeur 1
gen.close() # Fermeture du générateur
```

Le docstring : normes

PEP 257 : "Docstring Conventions"

- Le docstring commence et se termine par trois guillemets.
- La première ligne est une description de la fonction.
- Chaque paramètre de la fonction est décrit sur une nouvelle ligne, précédé de **:param** suivi du nom du paramètre, puis d'une description du paramètre.
- Si la fonction renvoie quelque chose, vous pouvez également spécifier cela avec **:return:** suivi d'une description de ce qui est retourné.
- Les autres sections facultatives peuvent inclure **:raises:** pour les exceptions levées par la fonction, **:rtype:** pour le type de retour, etc.

Le docstring : exemple

```
def saluer(nom, salutation="Bonjour", ponctuation="!"):
    """
    Fonction qui affiche une salutation personnalisée.

    :param nom: Le nom de la personne à saluer.
    :param salutation: La salutation à afficher (par défaut: "Bonjour").
    :param ponctuation: La ponctuation à ajouter à la fin de la salutation (par défaut: "!").
    :return: Aucun retour, affiche simplement la salutation.
    """
    print(salutation + ", " + nom + ponctuation)
```

La methode help()

La méthode **help()** est une fonction utilisée pour obtenir de l'aide sur les modules, les fonctions, les classes, etc. Elle fournit des informations sur leur utilisation, leur syntaxe et leurs arguments en affichant dans la console le commentaire docstring lié à l'élément fourni en paramètre. Exemple : **help(nom_fonction)**

Les annotations de type

PEP 484 : "Type Hints"

Type hints

- Une annotation de type (type hints) en Python est une fonctionnalité introduite pour permettre aux développeurs d'indiquer le type de données attendu pour les paramètres de fonction et les valeurs de retour.

✓ Cela permet d'améliorer la lisibilité du code et facilite la détection d'erreurs potentielles lors de la phase de développement.

→ Les annotations de type ne modifient pas le comportement du code Python et ne sont pas vérifiées lors de l'exécution, mais elles sont utilisées par certains outils d'analyse statique et de vérification de type pour détecter les incohérences de types potentielles.

Type hints : exemple (1/2)

type des paramètres type de retour

```
def addition(a: int, b: int) -> int:
    """
    Cette fonction effectue une addition entre deux entiers.

    :param a: Le premier entier.
    :param b: Le deuxième entier.
    :return: La somme des deux entiers.
    """
    return a + b
```

- Lors de la déclaration d'une méthode, il est possible de spécifier le type de chaque paramètre en utilisant la syntaxe nom_paramètre: type.
- Le type de retour sera précisé à la fin des parenthèses via l'utilisation d'une flèche → type_de_retour

Type hints : exemple (2/2)

type des paramètes — valeur par défaut — type de retour

```
def saluer(nom: str, salutation: str = "Bonjour", ponctuation: str = "!") -> None:
    """
    Fonction qui affiche une salutation personnalisée.

    :param nom: Le nom de la personne à saluer.
    :param salutation: La salutation à afficher (par défaut: "Bonjour").
    :param ponctuation: La ponctuation à ajouter à la fin de la salutation (par défaut: "!").
    :return: Aucun retour, affiche simplement la salutation.
    """
    print(salutation + ", " + nom + ponctuation)
```

- Pour tous les paramètres optionnels, la valeur par défaut doit suivre l'annotation de type.
- En Python lorsqu'il s'agit d'une procédure, le type de retour est **None** et non void.

Les types hints particulier : Set et List

- Les sets

type du paramètre — type de retour

```
def process_set(data: set[float]) -> set[float]:
```

type contenu dans la collection

- Les listes

type du paramètre — type de retour

```
def process_list(data: list[int]) -> list[int]:
```

type contenu dans la collection

- Dans chaque cas, vous devrez spécifier le type des variables contenu dans la collection.

Les types hints : Any et Union ?

Any / Union

Les listes en Python sont des types particuliers pouvant contenir plusieurs objets du même type ou de types différents. Pour annoter ces types, plusieurs approches sont possibles :

1. Votre liste peut contenir des éléments de n'importe quel type.
   → Utilisez le type hint : Any

```
def process_list(data: list[any]) -> list[any]:
```

2. Votre liste contient des types différents mais restreints.
   → Utilisez Union et précisez chacun de ces types.

```
from typing import Union
# Liste pouvant contenir des entiers ou des nombres à virgule flottante
def process_list(data: list[Union[int, float]]) -> list[Union[int, float]]:
```

importation

Les types hints particulier : Tuple et Dict

- Les tuples

type des paramètres — type du retour

```
def process_tuple(data: tuple[str, int]) -> tuple[str, int]:
```

type 1er élément — type 2e élément

- Les dictionaires

```
def process_dict(data: dict[str, int]) -> dict[str, int]:
```

type des clés — type des valeurs

- Si tous les éléments de votre tuple sont du même type, utilisez la notation : *Tuple[votre_type, ...]*. Sinon, vous devrez spécifier le type de chaque élément qui le compose.
- Pour les dictionnaires, le type des clés et des valeurs devront être précisé.

Les types hints particulier

```
# Python 3.9 et +
def process_list(data: list[int]) -> list[int]:

# Python 3.8 et -
from typing import List
def process_list(data: List[int]) -> List[int]:
```

Python >= 3.9 / Python < 3.9

- Pour les versions de Python supérieures ou égales à 3.9, vous pourrez utiliser ces annotations comme illustré dans ces exemples.
- Pour les versions antérieures à 3.9, vous devrez importer chaque annotation à partir du module typing. De plus, toutes ces annotations commenceront par une lettres majuscules.

Les types hints lié aux générateurs

```
from typing import Generator
```
importation

```
def process_data(data: list[int]) -> Generator[int, None, None]:
    for item in data:
        yield item * 2
```
yield send return

Il est à noter que le type d'un générateur est une collection de trois valeurs distinctes :

- yield: désigne le type des données produites par le générateur renvoyé via l'instruction **yield**.
- send: représente le type des données transmises au générateur pendant son exécution.
- return: indique le type des données renvoyées par le générateur via l'instruction **return**.

Début TP1

TP1 - Générateurs

- Développez un générateur qui produira des nombres premiers en utilisant comme paramètres un début, une fin et un pas. Comme pour range la borne de fin sera exclu.
  ✓ generateur(1, 5, 1) → 2, 3, 5, 7
  ✓ generateur(3, 5, 1) → 5, 7
  ✓ generateur(1, 5, 2) → 2, 5

- Développez un générateur calculant la suite de Fibonacci en utilisant les mêmes paramètres que le précédent.
  ✓ generateur(1, 10, 1) → 0, 1, 1, 2, 3, 5, 8, 13, 21
  ✓ generateur(5, 10, 1) → 3, 5, 8, 13, 21
  ✓ generateur(1, 10, 2) → 0, 1, 3, 8, 21

Plan du Chapitre

1. Les méthodes et la documentation
2. Les importations
3. L'avant POO
4. La programmation orientée objet

Les importations !

Les importations sont utilisés pour inclure dans votre programme actuel des modules ou des fonctions définies dans d'autres fichiers. La portée de ces fonctions ou éléments **est limitée au fichier dans lequel ils sont importés**. Cela vous permet d'utiliser du code réutilisable et d'organiser votre code de manière modulaire.

- **Importation de module:** *import nom_du_module*
- **Importation de fonction:** *from nom_du_module import nom_de_la_fonction*
- **Importation avec renommage de module:** *import nom_du_module as nouveau_nom*

Exemples d'importation (1/2)

- **Importation de module:**

```
import math

# Utilisation d'une fonction du module math
print(math.sqrt(25))  # Affiche la racine carrée de 25
```

- **Importation de fonction:**

```
from math import sqrt

# Utilisation de la fonction directement sans préfixe
print(sqrt(25))  # Affiche la racine carrée de 25
```

- **Importation avec renommage de module:**

```
import math as m

# Utilisation du module renommé
print(m.sqrt(25))  # Affiche la racine carrée de 25
```

Exemples d'importation (2/2)

Importation multiple

- Il est possible d'effectuer des importations multiples en séparant vos méthodes par des virgules dans la déclaration d'importation.

```
from math import sqrt, sin, cos
```

from module import *

- Même si la notation **import *** existe en Python elle est fortement déconseillée car elle peut entraîner des conflits de noms et rendre le code moins lisible.

Les espaces de noms (namespace)

espace de noms / namespace

Un espace de noms est un système qui garantit que les noms de variables sont uniques et identifiables dans un programme. Cela permet d'éviter les conflits de noms et de fournir un contexte clair.

- **Espace de noms local (local namespace)**
- **Espace de noms global (global namespace)**
- **Espace de noms intégré (built-in namespace)**

Installer un nouveau module ?

PIP

pip est un gestionnaire de paquets qui vous permet d'installer, de désinstaller et de gérer des paquets Python supplémentaires qui ne sont pas inclus dans la distribution standard.

- **Installer un paquet:** *pip install nom_du_package*
- **Installer une version précise:** *pip install nom_du_package==version*
- **Mettre à jour un paquet:** *pip install –upgrade nom_du_package*
- **Desinstaller un paquet:** *pip uninstall nom_du_package*
- **Lister les paquets installés:** *pip list*

Conflit : pip2 & pip3

Attention !

Si vous avez plusieurs versions de Python sur votre machine, notamment Python 2 et Python 3, vous avez également plusieurs versions de pip (`pip2` / `pip3`), chacune associée à la version correspondante de Python.

1. Assurez-vous de préciser à chaque requête la version de pip : `pip3`
2. Vous pouvez configurer un alias dans votre fichier `~/.bashrc` en ajoutant la ligne suivante : `alias pip='pip3'`

→ Pour afficher la version de `pip` par défaut, exécutez la commande suivante dans un terminal : `pip --version`

Un paquet ?

paquet / package

Dans le contexte de Python, un "package" fait référence à une structure d'organisation des modules. Un package est essentiellement un répertoire (ou un dossier) qui contient un fichier spécial appelé __init__.py ainsi que d'autres modules Python ou sous-packages.

```
my_package/
│
├── __init__.py
├── module1.py
├── module2.py
└── subpackage/
    ├── __init__.py
    ├── submodule1.py
    └── submodule2.py
```

- Les répertoires my_package et subpackage sont considérés comme un package Python car ils contiennent un fichier __init__.py.
- Les autres fichiers sont considérés comme des modules ou des sous-modules.

Connaître son environnement de travail

pip freeze

pip freeze est une commande utilisée pour générer une liste des paquets installés dans un environnement virtuel ou global, ainsi que leurs versions. Cette liste est généralement utilisée pour sauvegarder l'état des dépendances d'un projet Python afin de pouvoir le reproduire sur un autre système ou de partager facilement les exigences du projet avec d'autres développeurs.

- **Afficher la liste des paquets:** *pip freeze*
- **Enregistrer cette liste dans un fichier:** *pip freeze > requirements.txt*
- **Installer les paquets à partir d'un fichier:** *pip install -r requirements.txt*

Plan du Chapitre

1. Les méthodes et la documentation
2. Les importations
3. L'avant POO
   - Les fichiers
   - La gestion des erreurs
4. La programmation orientée objet

Les fichiers !

Les fichiers

Les fichiers sont des structures de données qui peuvent contenir une variété d'informations, telles que du texte, des images, du code, et bien d'autres encore.

Convention de nommage

La PEP 8, qui définit les conventions de style pour le code Python, recommande d'utiliser la convention de nommage snake_case également pour les noms de fichier. Cela signifie que les noms de fichiers doivent commencer par une minuscule et que les différents mots composant le titre doivent être séparés par des underscores.

Ouvrir un fichier (Open vs with Open)

La fonction **open()** est utilisée pour ouvrir un fichier. Elle prend en paramètre le chemin du fichier à ouvrir ainsi que le mode d'ouverture. Une fois le fichier traité celui-ci doit être fermé.

le fichier fonction chemin du fichier type d'ouverture

```
fichier = open("exemple.txt", "r")
# Instructions
fichier.close()
```

L'instruction **with open()** est une syntaxe recommandée pour ouvrir et gérer des fichiers. Elle crée un contexte de gestion de fichier qui garantit que le fichier est correctement fermé à la fin, même en cas d'erreur pendant le traitement.

mot clé fonction chemin du fichier type d'ouverture le fichier

```
with open("exemple.txt", "w") as fichier: 2 points
    # Instructions
```

Les types d'ouverture (1/2)

- "r" (read) : Ouvre le fichier en mode lecture. Le fichier doit exister, sinon une erreur sera levée.
- "w" (write) : Ouvre le fichier en mode écriture. Si le fichier existe, son contenu sera supprimé. Si le fichier n'existe pas, il sera créé.
- "a" (append) : Ouvre le fichier en mode ajout. Les nouvelles données seront ajoutées à la fin du fichier, sans supprimer le contenu existant. Si le fichier n'existe pas, il sera créé.

Les types d'ouverture (2/2)

- "x" (exclusive creation) : Ouvre le fichier en mode création exclusive. Si le fichier existe déjà, une erreur sera levée.
- "b" (binary) : Ouvre le fichier en mode binaire, ce qui est utile pour les fichiers non textuels (comme les images ou les fichiers binaires). Ce mode est souvent utilisé en combinaison avec les modes "r", "w" ou "a".
- "+" (read/write) : Ouvre le fichier en mode lecture et écriture. Ce mode est généralement utilisé en combinaison avec les modes "r", "w" ou "a".

Méthodes liées aux fichiers

Méthodes	**Description**
close()	Ferme le fichier.
open(path)	Ouvre le fichier situé au chemin spécifié.
read()	Lit le fichier entier.
readline()	Lit et retourne la première ligne du fichier sous forme de chaîne de caractères.
readlines()	Lit et retourne une liste contenant toutes les lignes du fichier.
write(string)	Écrit la chaîne de caractères spécifiée dans le fichier.
writelines(lines)	Écrit une liste de lignes dans le fichier.

Erreurs / Exceptions

Exception

Les exceptions sont des événements qui se produisent lorsqu'une condition inattendue ou indésirable se produit pendant l'exécution d'un programme. Lorsqu'une exception est levée, le programme peut soit la gérer pour continuer son exécution de manière contrôlée, soit laisser l'exception se propager, ce qui arrête l'exécution du programme et affiche un message d'erreur.

Les mots clés liés aux exceptions

Le traitement des exceptions

1. Lever une exception : Vous pouvez lever une exception à l'aide du mot-clé **raise**.

```
raise ValueError("Valeur incorrecte")
```

2. Gérer les exceptions : Vous pouvez gérer les exceptions à l'aide de blocs **try**, **except**, **else** et **finally**.

```
try:
    # Code susceptible de lever une exception
except SomeException:
    # Code pour gérer l'exception
else:
    # Code à exécuter si aucune exception n'est levée
finally:
    # Code à exécuter quel que soit le résultat
```

La gestion d'exception : explication

mot clé

```
try:
    # Tentative d'ouverture d'un fichier
    fichier = open("exemple.txt", "r")
    # Lecture du contenu du fichier
    contenu = fichier.read()
    print(contenu)
```

nom de l'exception

```
except FileNotFoundError:
    # Gestion de l'exception si le fichier n'est pas trouvé
    print("Le fichier spécifié est introuvable.")
```

```
else:
    # Code à exécuter si aucune exception n'est levée
    print("Lecture du fichier terminée avec succès.")

finally:
    # Code à exécuter quel que soit le résultat
    if 'fichier' in locals():
        # Fermeture du fichier s'il est ouvert
        fichier.close()
        print("Fermeture du fichier.")
    else:
        print("Aucun fichier n'a été ouvert.")
```

bloc d'instructions optionel

Le bloc try / else

Le bloc try

Dans un bloc **try**, vous n'êtes pas obligé d'encapsuler uniquement les instructions qui pourraient lever une exception. Cependant, en Python, il est recommandé d'utiliser le bloc **else** dans de tels cas.

le bloc else

Le bloc **else** en Python est utilisé en conjonction avec le bloc **try**. Il est exécuté lorsque aucune exception n'est levée dans le bloc **try**. En d'autres termes, si l'exécution du code dans le bloc **try** se déroule sans rencontrer d'erreurs, alors le bloc **else** est exécuté immédiatement après le bloc **try**. Cela permet d'ajouter des instructions supplémentaires à exécuter lorsque le code dans le bloc **try** réussit sans problème.

Le bloc except (1/2)

L'instruction except

Après l'instruction **except**, vous avez la possibilité de regrouper plusieurs types d'exceptions afin de les traiter de la même manière en les plaçant dans un tuple. De plus, vous pouvez stocker l'exception générée dans une variable en utilisant le mot-clé **as**.

```
try:
    # Code susceptible de lever différentes exceptions
    result = x / y

                    les erreurs gérées          variable contenant l'exception générée
except (ZeroDivisionError, TypeError, ValueError) as e:
    # Gérer les exceptions ZeroDivisionError, TypeError et ValueError
    print("Une erreur s'est produite :", e)
```

La mère de toutes les exceptions !

"De grands pouvoirs impliquent de grandes responsabilités !"

– Oncle Ben (Spiderman)

BaseException

- Dans un bloc **except**, ne spécifiez que les types d'exceptions que vous allez gérer !

  Il est donc formellement interdit de spécifier le type BaseException, qui regroupe toutes les erreurs et exceptions existantes en Python. De même pour le type Exception.

Le bloc except (2/2)

L'instruction except

Vous pouvez également cumuler plusieurs blocs **except** dans le cas où vous souhaitez traiter vos exceptions de manière différente.

```
try:
    x = int(input("Entrez un nombre : "))
    result = 10 / x
    print("Le résultat est :", result)
except ValueError:
    print("Vous devez entrer un nombre entier.")
except ZeroDivisionError:
    print("Vous ne pouvez pas diviser par zéro.")
```

✓ Bonne pratique

Lorsque vous affichez une erreur à l'écran, veillez toujours à l'accompagner d'un message explicatif clair, afin d'expliquer à l'utilisateur ce qu'il a mal fait. Cela permet de lui éviter de réitérer la même erreur.

Les exceptions personnalisées !

- Créer son propre type d'exception :

```
class MonException(Exception):
    def __init__(self, message="Une erreur personnalisée s'est produite"):
        self.message = message
        super().__init__(self.message)
```

- La propager :

```
def fonction_critique(parametre):
    if parametre < 0:
        raise MonException("Le paramètre ne peut pas être négatif")
```

mot clé nom de l'exception Message d'erreur

N'oubliez pas d'importer votre exception si nécessaire.

Plan du Chapitre

1. Les méthodes et la documentation
2. Les importations
3. L'avant POO
4. La programmation orientée objet
   - L'objet
   - L'héritage
   - Le polymorphisme

Structure d'un code python !

```python
"""
Ce fichier/module est utilisé pour effectuer des opérations mathématiques simples.
"""

# Imports de la bibliothèque standard de Python
import os
import sys

# Imports des modules externes (third-party)
import numpy as np
import pandas as pd

# Imports des modules locaux ou spécifiques à l'application
from my_module import my_function

# Variables globales
DEBUG = True
MAX_ITERATIONS = 1000

# Définition des fonctions/classes
def add(a, b):
    return a + b

class MyClass:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"Hello, {self.name}!")

# Code exécutable
if __name__ == "__main__":
    my_obj = MyClass("Alice")
    my_obj.greet()
```

Importation

Variables globales

Méthodes / Classe

Main

La Programmation Orientée Objet

Les concepts clés :

- Classes et Objets
- Attributs et Méthodes
- Encapsulation
- Héritage
- Polymorphisme

Classe & Encapsulation

La classe

- Une classe définit un modèle pour la création d'objets. Elle encapsule des données (attributs) et des fonctions (méthodes) qui agissent sur ces données.

→ Il est recommandé de n'avoir qu'une seule classe par fichier !

mot clé nom 2 points

```
class Personne:
```

variable de la classe

```
    nombre_instance: int = 0
```

Méthodes de la classe

```
    def __init__(self, nom, age):
        self.nom: str = nom
        self.age: int = age
        Personne.nombre_instances += 1

    def presenter(self):
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")
```

La construction d'instance !

Le constructeur

- La méthode __init__() est le constructeur d'une classe. Elle est appelée automatiquement à chaque fois qu'une nouvelle instance de la classe est créée.

mot clé constructeur paramètres

```
def __init__(self, nom, age):
    self.nom = nom
    self.age = age
    Personne.nombre_instances += 1
```

2 points

instructions

Objet / Instance

Instance de classe

- Les objets sont des instances spécifiques d'une classe et représentent des entités du monde réel ou des concepts abstraits.

- Création de plusieurs objets de la classe Personne

```
aimane = Personne("Aimane", 27)
ange = Personne("Ange", 19)
```

- Récupération du type de l'objet "aimane"

mot clé variable

```
type_aimane = type(aimane)
print(type_aimane) # Personne
```

- L'objet "ange" est-il une instance de la classe Personne ?

mot clé variable type

```
is_ange_personne = isinstance(ange, Personne)
print(is_ange_personne) # True
```

self ?

Le mot-clé self n'est pas utilisé lors de la création d'une instance !

Mais quel est son rôle ?

self : le this de Python !

Le mot clé self

- Le rôle de **self** est de permettre à la méthode d'accéder aux attributs et aux méthodes de l'objet sur lequel elle est appelée. Self est une référence à l'instance elle-même, et sera passée implicitement lors de l'appel à la méthode.

  Cela signifie que lorsque vous définissez une méthode dans une classe, vous devez inclure **self** comme premier paramètre dans la signature de la méthode pour indiquer qu'elle est une méthode de l'instance.

Le mot clé self

```
class Personne:

    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def presenter(self):
                  [alice]
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")
                             [alice]              [alice]

# Création d'un objet personne
alice = Personne( "Alice", 30)

alice.presenter()
```

Figure: Explication du fonctionnement du mot clé self

__str__() le toString() de Python

La méthode **__str__()** en Python est une méthode spéciale utilisée pour définir une représentation sous forme de chaîne de caractères d'un objet. Lorsqu'elle est définie dans une classe, elle permet de personnaliser la façon dont les instances de cette classe sont converties en chaînes de caractères lorsqu'elles sont utilisées dans des fonctions de conversion comme **str()** ou lorsqu'elles sont affichées avec **print()**.

```python
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def __str__(self):
        return f"Personne: {self.nom}, {self.age} ans"

# Création d'une instance de la classe Personne
fabrice = Personne("Fabrice", 19)

# Utilisation de la méthode __str__ pour obtenir une représentation sous forme de chaîne de caractères
print(str(fabrice))  # Affiche : Personne: Fabrice, 19 ans

# La fonction print() appelle également implicitement la méthode __str__
print(fabrice)  # Affiche : Personne: Fabrice, 19 ans
```

Les méthodes propre aux objets : __eq__ et __hash__

__eq__

La méthode __eq__ est utilisée pour définir le comportement de l'opérateur d'égalité (==) entre les instances d'une classe. Elle prend en paramètre l'objet avec lequel on souhaite comparer l'instance et retourne True si les deux objets sont considérés comme égaux, sinon False.

__hash__

La méthode __hash__ est essentielle pour fournir un code de hachage unique à un objet, permettant ainsi son utilisation dans des structures de données basées sur le hachage comme les dictionnaires et les ensembles. Elle retourne un entier représentant ce hachage. Il est crucial que des objets considérés comme égaux renvoient le même code de hachage.

__eq__ et __hash__ : Exemple

```python
class Personne:
    def __init__(self, prenom, nom, age):
        self.prenom = prenom
        self.nom = nom
        self.age = age

    def __str__(self):
        return f"Personne: {self.nom}, {self.prenom}, {self.age} ans"

    def __eq__(self, other):
        return self.nom == other.nom and self.prenom == other.prenom and self.age == other.age

    def __hash__(self):
        # Utilisation d'un hash simple en combinant les hashs des attributs
        return hash((self.nom, self.prenom, self.age))


# Création de deux personnes
p1 = Personne("Leny", "Denoyelle", 19)
p2 = Personne("Jordan", "Horbacz", 19)

# Test d'égalité
print(p1 == p1)  # Affiche True
print(p1 == p2)  # Affiche False
```

- Si vous définissez **__eq__**, vous devriez également définir **__hash__** pour garantir que les objets qui sont égaux ont le même code de hachage.

Les méthodes propres aux objets

len, getitem, setitem, delitem et contains

Sont des méthodes vous permettant de traiter vos objets comme des collections. En implémentant ces méthodes dans votre classe, vous pouvez rendre votre objet compatible avec les opérations courantes sur les collections, comme l'accès par index, la modification, la suppression, la vérification d'appartenance...

Pour connaître toutes les méthodes propres aux objets, consultez la documentation officielle.

len, getitem, setitem, delitem et contains

```python
class CustomList:
    def __init__(self):
        self._data = []

    def __getitem__(self, index):
        return self._data[index]

    def __setitem__(self, index, value):
        self._data[index] = value

    def __delitem__(self, index):
        del self._data[index]

    def __len__(self):
        return len(self._data)

    def __contains__(self, item):
        return item in self._data
```

```python
if __name__ == "__main__" :

    # Exemple d'utilisation
    custom_list = CustomList()
    custom_list.append(1)

                    __getitem__
    print(custom_list[0])

        __len__
    print(len(custom_list))

        __contains__
    print(2 in custom_list)

                __setitem__
    custom_list[0] = 0

__delitem__
    del custom_list[1]
```

Les méthodes propres aux objets : avertissement

Avertissement !

- Lorsque vous définissez une classe, seules deux méthodes sont héritées par défaut : la méthode __init__() et la méthode __repr__().
- La première est utilisée pour initialiser les attributs de l'objet lors de sa création, tandis que la seconde renvoie une représentation de chaîne non ambiguë de l'objet.
- Si vous appelez d'autres méthodes spécifiques à l'objet qui n'ont pas été implémentées, cela entraînera une erreur.

La surcharge d'opérateur ?

Connaissez-vous le concept de surcharge d'opérateur ?

Si oui, pourriez-vous le définir ?

La surcharge d'opérateur !

La surcharge d'opérateur

- La surcharge d'opérateur consiste à redéfinir le comportement des opérateurs intégrés (+, -, *, /, etc.) pour les objets de vos propres classes. En redéfinissant les méthodes spéciales correspondantes **__add__, __sub__, __mul__ , __truediv__**, etc.. vous pouvez permettre à vos objets d'utiliser ces opérateurs de manière intuitive, comme s'ils étaient des types de données intégrés.
  Parmi ces méthodes, on trouve également celles correspondant aux opérations d'affectation telles que +=, -=, *=, /=, etc.

Surcharge d'opérateur : exemple

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __sub__(self, other):
        if isinstance(other, Point):
            dx = self.x - other.x
            dy = self.y - other.y
            return Point(dx, dy)
        else:
            raise TypeError("Subtraction operation not supported between Point and non-Point objects.")


    def __str__(self):
        return f"({self.x}, {self.y})"


# Exemple d'utilisation
p1 = Point(5, 3)
p2 = Point(2, 1)

difference = p1 - p2

print(difference)  # Affiche la différence entre les points
```

Les méthodes de classe et les méthodes statiques (1)

Méthode de classe

- Ce sont les méthodes qui prennent implicitement l'instance de la classe (self) en premier paramètre.
- Elles ont accès aux attributs de l'instance via self.
- Elles sont utilisées pour manipuler les attributs spécifiques à chaque instance de la classe.

Méthode statique

- Ce sont des méthodes qui n'ont pas besoin de l'instance de la classe comme premier paramètre.
- Elles sont décorées avec @staticmethod.
- Elles sont utilisées lorsque la méthode ne dépend pas de l'état de l'instance, mais qu'elle est logiquement liée à la classe.

Les méthodes de classe et les méthodes statiques (2)

```
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
```

Constructeur

```
    def presenter(self):
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")
```

Méthode d'instance

```
    @staticmethod
    def est_adulte(age):
        return age >= 18
```

Méthode statique

Les décorateurs

- Un décorateur est une fonction qui prend une autre fonction en tant qu'argument et retourne une fonction modifiée ou une version améliorée de cette fonction. En d'autres termes, les décorateurs permettent de modifier le comportement d'une fonction sans modifier son code interne.
- La syntaxe de base d'un décorateur consiste à placer un "@" suivi du nom du décorateur au-dessus de la fonction à décorer. Cela permet d'appliquer le décorateur à cette fonction.
- Il est possible d'appliquer plusieurs décorateurs à une même fonction en les empilant les uns au-dessus des autres.

→ Attention quand même à ne pas perdre en lisibilité

Une histoire de portée ?

Seriez-vous capable de définir les 3 portées utilisées en programmation orientée objet ?

Les portées : public, private et protected

- **La portée publique :** l'élément peut être accessible depuis n'importe où en dehors de la classe.
- **La portée protégée :** l'élément n'est accessible que depuis la classe ou les classes héritant de celle-ci. En Python sont considérés comme protégé les éléments définis avec un underscore comme préfixe (_)
- **La portée privée :** l'élément ne peut pas être accédé ou modifié depuis l'extérieur de la classe. En Python sont considérés comme privé les éléments définis avec deux underscores comme préfixe (__)

Les portées en Python

```python
class MyClass:
    def __init__(self):
        self.public_attribute = "public"  # Portée publique
        self._protected_attribute = "protected"  # Portée protégée
        self.__private_attribute = "private"  # Portée privée

    def public_method(self):
        print("Public method called")

    def _protected_method(self):
        print("Protected method called")

    def __private_method(self):
        print("Private method called")
```

Attention !

En Python, par défaut, tous les membres d'une classe (attributs et méthodes) sont considérés comme publics.

Les importations de classe

- Importer une classe d'un autre fichier :
  from mon_fichier import MaClasse
- Importer une classe depuis un répertoire différents :
  from ..dossier_principal.mon_fichier import MaClasse

→ Si le fichier contenant la classe se trouve dans un répertoire différent, vous pouvez spécifier le chemin relatif ou absolu vers ce fichier lors de l'importation en utilisant le point (.) pour indiquer le chemin relatif à partir du répertoire actuel ou double point (..) pour indiquer le répertoire parent.

X Il est important de noter que l'importation est locale au fichier dans lequel elle est effectuée, ce qui signifie que si vous avez besoin d'utiliser la classe dans plusieurs fichiers, vous devrez l'importer dans chaque fichier où vous en avez besoin.

Les annotations liées à vos classes

```python
class Point:
    """
    Classe représentant un point dans un plan cartésien.

    Attributes:
        x (float): La coordonnée x du point.
        y (float): La coordonnée y du point.
    """

    def __init__(self, x: float, y: float) -> None:
        """
        Initialise un point avec les coordonnées spécifiées.

        Args:
            x (float): La coordonnée x du point.
            y (float): La coordonnée y du point.
        """
        self.x: float = x
        self.y: float = y

    def __repr__(self) -> str:
        """
        Retourne une représentation sous forme de chaîne de caractères du point.

        Returns:
            str: La représentation du point.
        """
        return f"Point({self.x}, {self.y})"
```

```python
class Figure:
    """
    Classe représentant une figure composée de plusieurs points.
    """

    def __init__(self) -> None:
        """
        Initialise une figure vide sans aucun point.
        """
        self.points: List[Point] = []

    def ajouter_point(self, point: Point) -> None:
        """
        Ajoute un point à la figure.

        Args:
            point (Point): Le point à ajouter à la figure.
        """
        self.points.append(point)

    def afficher_points(self) -> None:
        """
        Affiche les points de la figure.
        """
        print("Points de la figure:")
        for point in self.points:
            print(point)
```

- Lorsque vous créez une classe, il est important de spécifier les type hints pour chacun de vos attributs. Si vous utilisez des types personnalisés, il vous suffira de les préciser en indiquant le nom de leur classe.

L'héritage en POO

L'héritage est un concept clé de la programmation orientée objet (POO) qui permet à une classe (appelée classe fille) d'hériter des attributs et des méthodes d'une autre classe (appelée classe mère). Cela permet la réutilisation du code et la mise en œuvre de la relation "est-un".

- **La classe mère (super classe) :** destinée à léguer toutes ses variables et méthodes marquées comme protégées.
- **La classe fille (sous classe) :** hérite des attributs et des méthodes de la classe mère. Elle peut également ajouter de nouveaux attributs et méthodes ou modifier ceux hérités.

→ En Python, pour hériter d'une classe mère, il suffit d'indiquer son nom entre parenthèses après le nom de votre classe.

Les classes filles

```
class Animal:
    def __init__(self, nom):
        self.nom = nom

class Chien(Animal):
    def __init__(self, nom, race):
        super().__init__(nom)
        self.race = race
```

Classe mère

super()

- Le mot-clé super est utilisé pour appeler les méthodes de la classe parente à partir de la classe enfant : **super().methode_classe_mere()**. Si celui-ci est appelé seul il fait référence au constructeur de la classe mère.
- Si vous ne définissez pas de constructeur à votre classe fille celui de la classe mère sera automatiquement utilisé lors de la création d'une instance.

L'héritage en Python

```python
class Animal:
    def __init__(self, nom):
        self.nom = nom

    def manger(self):
        print(f"{self.nom} mange")
```

classe mère

```python
class Chien(Animal):
    def aboyer(self):
        print(f"{self.nom} aboie")
```

→ Hérite de la méthode manger(self)

classe mère

```python
class Chat(Animal):
    def miauler(self):
        print(f"{self.nom} miaule")
```

→ Hérite de la méthode manger(self)

```python
if __name__ == "__main__" :
    chien = Chien("Max")
    chien.manger()  # Appel de la méthode de la classe Animal
    chien.aboyer()  # Appel de la méthode de la classe Chien

    chat = Chat("Minou")
    chat.manger()   # Appel de la méthode de la classe Animal
    chat.miauler()  # Appel de la méthode de la classe Chat
```

L'héritage multiple

```python
class Pere:
    def parler(self):
        print("Je suis le père")


class Mere:
    def parler(self):
        print("Je suis la mère")

class Enfant(Pere, Mere):
    def parler(self):
        print("Je suis l'enfant")
```

Héritage multiple

```python
if __name__ == "__main__" :

    # Création d'une instance de la classe Enfant
    enfant = Enfant()

    # Appels de la méthode parler()
    enfant.parler()  # Affiche : Je suis l'enfant
```

L'héritage multiple

- En Python, il est possible d'implémenter l'héritage multiple en spécifiant plusieurs classes parentes lors de la création d'une nouvelle classe, en les séparant par des virgules.

Problème d'héritage ?

Comment faire lorsqu'on hérite de plusieurs classes ayant chacune une définition de la même méthode ?

MRO (Method Resolution Order)

Pour éviter le problème du diamant (lorsque deux classes héritent chacune d'une même classe de base, et une troisième classe hérite de ces deux classes dérivées), Python utilise l'ordre de résolution des méthodes (MRO). L'ordre MRO détermine l'ordre dans lequel les méthodes sont recherchées lorsqu'elles sont appelées sur une instance de classe. Il commence par vérifier si la méthode est définie dans la classe elle-même. Si ce n'est pas le cas, il recherche dans la première classe parente, puis dans la deuxième, et ainsi de suite.

Les classes abstraites (1/4)

Pour déclarer une classe abstraite en Python : Vous pouvez importer le module **abc** (Abstract Base Classes). Ce qui vous permettra de déclarer des méthodes abstraites qui seront à définir dans les classes filles. Chacune des méthodes abstraites devra être précédée du décorateur **@abstractmethod** et contenir l'instruction **pass**.

```
from abc import ABC, abstractmethod

class Forme(ABC):
    @abstractmethod
    def aire(self):
        pass

class Rectangle(Forme):
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    def aire(self):
        return self.longueur * self.largeur
```

Les classes abstraites (2/4)

Remarquez-vous quelque chose de particulier dans ce code ?

Est-ce que quelque chose vous semble inhabituel ?

```
from abc import ABC, abstractmethod

class Forme(ABC):
    @abstractmethod
    def aire(self):
        pass

class Rectangle(Forme):
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    def aire(self):
        return self.longueur * self.largeur
```

Les classes abstraites (3/4)

Python gère nativement les classes abstraites. :9

```
class Forme():
    def aire(self):
        pass

class Rectangle(Forme):
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    def aire(self):
        return self.longueur * self.largeur
```

- Comme illustré dans cet exemple, le code est similaire au précédent, mais n'utilise ni module ni décorateur. Python gère nativement les méthodes abstraites grâce à l'instruction **'pass'**.

✓ Il donc est recommandé de privilégier cette syntaxe !

Les classes abstraites (4/4)

2 syntaxes ?

- Pourquoi existe-t-il deux syntaxes différentes ?

→ Tout simplement pour faciliter la transition des développeurs venant d'autres langages tels que Java.

Avertissement !

- Attention, ni l'utilisation de l'annotation @abstractmethod ni la déclaration de méthodes abstraites sans cette annotation ne garantissent explicitement l'implémentation de ces méthodes dans les sous-classes.

Le polymorphisme

Le polymorphisme est un principe de la programmation orientée objet où deux objets héritant de la même classe partagent un comportement commun mais peuvent l'exprimer chacun à leur manière.

- Polymorphisme ad hoc [X]: Il permet à une fonction ou méthode d'accepter différents types d'arguments et d'agir en conséquence. Cela peut être réalisé par surcharge de fonction ou par utilisation de paramètres génériques.
- Polymorphisme d'héritage [✓]: Il permet à des objets de différentes classes d'être traités de la même manière s'ils sont des instances de la même superclasse. Cela implique que les méthodes d'une classe parente peuvent être redéfinies dans les classes enfants pour fournir des fonctionnalités spécifiques à chaque classe.

Polymorphisme d'héritage

```python
class Animal :
    def faire_son(self):
        pass


class Chien(Animal):
    def faire_son(self):
        return "Woof !"

class Chat(Animal):
    def faire_son(self):
        return "Meow !"
```

```python
if __name__ == "__main__":

    animaux = []

    animaux.append(Chien())
    animaux.append(Chat())

    for animal in animaux :
        print(animal.faire_son())
```

Sources

- Geekforgeek
- Wikipedia
- ChatGPT

TP 2 à 4

- TP 2 : https://adventofcode.com/2017/day/15
- TP 3 (Objet demandé): https://adventofcode.com/2017/day/13
- TP 4 (Objet demandé): https://adventofcode.com/2017/day/20

TP Bonus (difficulté croissante)

- https://adventofcode.com/2018/day/2
- https://adventofcode.com/2017/day/11
- https://adventofcode.com/2021/day/6
- https://adventofcode.com/2020/day/21